

Experiment 8: Configure Kubernetes network policies to restrict traffic between pods.

Objectives

- Understand default pod communication
- Apply **NetworkPolicy**
- Restrict traffic between pods

Step 1: Verify Docker Installation

```
mkdir exp8 && cd exp8
docker --version
docker ps
```

```
cse@cse-B760M-H:~/exp8$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
cse@cse-B760M-H:~/exp8$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED    STATUS    PORTS    NAMES
```

Step 2: Verify kubectl

```
kubectl version --client
```

```
cse@cse-B760M-H:~/exp8$ kubectl version --client
Client Version: v1.35.3
Kustomize Version: v5.7.1
cse@cse-B760M-H:~/exp8$
```

Step 3: Verify Minikube

```
minikube version
```

```
cse@cse-B760M-H:~/exp8$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
cse@cse-B760M-H:~/exp8$
```

Step 4: Start Minikube

```
minikube start --driver=docker
```

Wait until:

Done! kubectl is now configured to use "minikube"

```
cse@cse-B760M-H:~/exp8$ minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b90d33ea03f75f2c4120e9b0
cse@cse-B760M-H:~/exp8$ minikube start --driver=docker
🐹 minikube v1.38.1 on Ubuntu 24.04
🌟 Using the docker driver based on user configuration
⚠️ Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
🔧 Using Docker driver with root privileges
👍 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
📦 Downloading Kubernetes v1.35.1 preload ...
> preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 16.10 M
> gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 15.03 M
🔥 Creating docker container (CPUs=2, Memory=3900MB) ...
🔧 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔗 Configuring bridge CNI (Container Networking Interface) ...
🔍 Verifying Kubernetes components...
  ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: storage-provisioner, default-storageclass
🏠 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
cse@cse-B760M-H:~/exp8$
```

Step 5: Verify Cluster

```
kubectl get nodes
```

Expected:

```
minikube Ready
```

```
cse@cse-B760M-H:~/exp8$ kubectl get nodes
NAME          STATUS    ROLES          AGE    VERSION
minikube      Ready    control-plane   71s    v1.35.1
cse@cse-B760M-H:~/exp8$
```

Step 6: Enable Network Policy Support

By default, **Minikube does not enforce network policies properly** unless a **CNI plugin** is enabled. We must enable **Calico** (recommended).

Enable Calico (One-time setup)

```
kubectl apply -f
https://docs.projectcalico.org/manifests/calico.yaml
```

```
cse@cse-B760M-H:~/exp8$ kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
poddisruptionbudget.policy/calico-kube-controllers created
serviceaccount/calico-kube-controllers created
serviceaccount/calico-node created
configmap/calico-config created
customresourcedefinition.apiextensions.k8s.io/bgpconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/bgppeers.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/blockaffinities.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/caliconodestatuses.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/clusterinformations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/felixconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/globalnetworksets.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/hostendpoints.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamblocks.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamconfigs.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ipamhandles.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippools.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/ippreservations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/kubecontrollersconfigurations.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networkpolicies.crd.projectcalico.org created
customresourcedefinition.apiextensions.k8s.io/networksets.crd.projectcalico.org created
clusterrole.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrole.rbac.authorization.k8s.io/calico-node created
clusterrolebinding.rbac.authorization.k8s.io/calico-kube-controllers created
clusterrolebinding.rbac.authorization.k8s.io/calico-node created
daemonset.apps/calico-node created
deployment.apps/calico-kube-controllers created
cse@cse-B760M-H:~/exp8$
```

Verify Calico is running:

```
kubectl get pods -n kube-system
```

Look for:

```
calico-node  
calico-kube-controllers
```

Status should be **Running**

```
cse@cse-B760M-H:~/exp8$ kubectl get pods -n kube-system  
NAME                                READY   STATUS    RESTARTS   AGE  
calico-kube-controllers-984995b45-vdwcj  1/1     Running   0           53s  
calico-node-jnn5r                      1/1     Running   0           53s  
coredns-7d764666f9-mnnh4              1/1     Running   0          2m34s  
etcd-minikube                          1/1     Running   0          2m40s  
kube-apiserver-minikube                 1/1     Running   0          2m40s  
kube-controller-manager-minikube        1/1     Running   0          2m41s  
kube-proxy-tl26k                       1/1     Running   0          2m34s  
kube-scheduler-minikube                 1/1     Running   0          2m40s  
storage-provisioner                     1/1     Running   1 (2m3s ago) 2m39s  
cse@cse-B760M-H:~/exp8$
```

```
minikube delete
```

```
minikube start --driver=docker --cni=calico
```

```
kubectl get nodes
```

Expected:

```
minikube Ready
```

```
cse@cse-B760M-H:~/exp8$ kubectl get nodes  
NAME          STATUS    ROLES          AGE   VERSION  
minikube      Ready     control-plane  71s   v1.35.1  
cse@cse-B760M-H:~/exp8$
```

Step 7: Test Basic Pod Deployment

Before starting Experiment 8, ensure pods work:

```
kubectl run test-pod --image=nginx
```

```
kubectl get pods
```

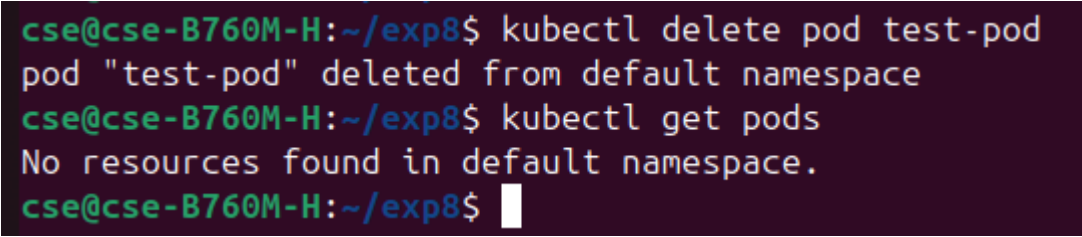
Status should be **Running**

```
cse@cse-B760M-H:~/exp8$ kubectl get pods  
NAME          READY   STATUS    RESTARTS   AGE  
test-pod      1/1     Running   0           10s  
cse@cse-B760M-H:~/exp8$
```

Step 8: Clean Up (Optional)

```
kubectl delete pod test-pod
```

```
kubectl get pods
```



```
cse@cse-B760M-H:~/exp8$ kubectl delete pod test-pod
pod "test-pod" deleted from default namespace
cse@cse-B760M-H:~/exp8$ kubectl get pods
No resources found in default namespace.
cse@cse-B760M-H:~/exp8$
```

Common Issue:

If Calico is not installed:

Network policies will be created

But **they will NOT be enforced**

So we will think policies are working, but they are not.

NOTE: “Can Kubernetes restrict traffic between pods by default?”

Answer:

No — by default all pods can communicate freely

*Network policies are used to **restrict communication***

Final Readiness Check

Run:

```
kubectl get nodes
```

```
kubectl get pods -A
```

Node → Ready

System pods → Running

```
cse@cse-B760M-H:~/exp8$ kubectl get nodes
NAME        STATUS    ROLES    AGE     VERSION
minikube    Ready     control-plane  5m42s   v1.35.1
cse@cse-B760M-H:~/exp8$ kubectl get pods -A
NAMESPACE   NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system  calico-kube-controllers-984995b45-vdwcj                1/1     Running   0           3m59s
kube-system  calico-node-jnn5r                                       1/1     Running   0           3m59s
kube-system  coredns-7d764666f9-mnnh4                              1/1     Running   0           5m40s
kube-system  etcd-minikube                                           1/1     Running   0           5m46s
kube-system  kube-apiserver-minikube                                1/1     Running   0           5m46s
kube-system  kube-controller-manager-minikube                       1/1     Running   0           5m47s
kube-system  kube-proxy-tl26k                                        1/1     Running   0           5m40s
kube-system  kube-scheduler-minikube                                1/1     Running   0           5m46s
kube-system  storage-provisioner                                     1/1     Running   1 (5m9s ago) 5m45s
cse@cse-B760M-H:~/exp8$
```

Now We Are Ready for Experiment 8

Our experiment will involve:

- Creating **multiple pods**
- Allowing traffic selectively
- Blocking traffic using **NetworkPolicy**

Precondition

Make sure:

- Minikube is running
- Calico is enabled (for network policy enforcement)

```
kubectl get nodes
```

```
kubectl get pods -n kube-system
```

Ensure Calico pods are running

Step 9: Create Two Pods

We will create:

- Pod A → frontend
- Pod B → backend

Create frontend pod

```
kubectl run frontend --image=nginx --labels=app=frontend
```

Create backend pod

```
kubectl run backend --image=nginx --labels=app=backend
```

```
cse@cse-B760M-H:~/exp8$ kubectl run frontend --image=nginx --labels=app=frontend
pod/frontend created
cse@cse-B760M-H:~/exp8$ kubectl run backend --image=nginx --labels=app=backend
pod/backend created
```

Verify pods

```
kubectl get pods
```

Both should be **Running**

```
cse@cse-B760M-H:~/exp8$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
backend       1/1     Running   0           45s
frontend      1/1     Running   0           53s
cse@cse-B760M-H:~/exp8$
```

Step 10: Test Communication (Default Behavior)

By default, Kubernetes allows all traffic.

Access frontend pod

```
kubectl exec -it frontend -- /bin/bash
```

Install curl (inside pod)

```
apt update && apt install curl -y
```

```
cse@cse-B760M-H:~/exp8$ kubectl exec -it frontend -- /bin/bash
root@frontend:/# apt update && apt install curl -y
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [114 kB]
Fetched 10.0 MB in 1s (14.0 MB/s)
1 package can be upgraded. Run 'apt list --upgradable' to see it.
curl is already the newest version (8.14.1-2+deb13u2).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1
root@frontend:/#
```

Get backend pod IP

In another terminal:

```
kubectl get pods -o wide
```

Note backend IP (example: 10.244.x.x)

```
cse@cse-B760M-H:~$ cd exp8/
cse@cse-B760M-H:~/exp8$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
backend	1/1	Running	0	2m32s	10.244.0.6	minikube	<none>		<none>	
frontend	1/1	Running	0	2m40s	10.244.0.5	minikube	<none>		<none>	

```
cse@cse-B760M-H:~/exp8$
```

Test connection

Inside frontend pod:

```
curl <backend-pod-ip>
```

We should see:

Welcome to nginx!

This proves:

Pods can communicate freely by default

```
root@frontend:/# curl 10.244.0.6
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@frontend:/#
```

Step 11: Create Network Policy (Restrict Traffic)

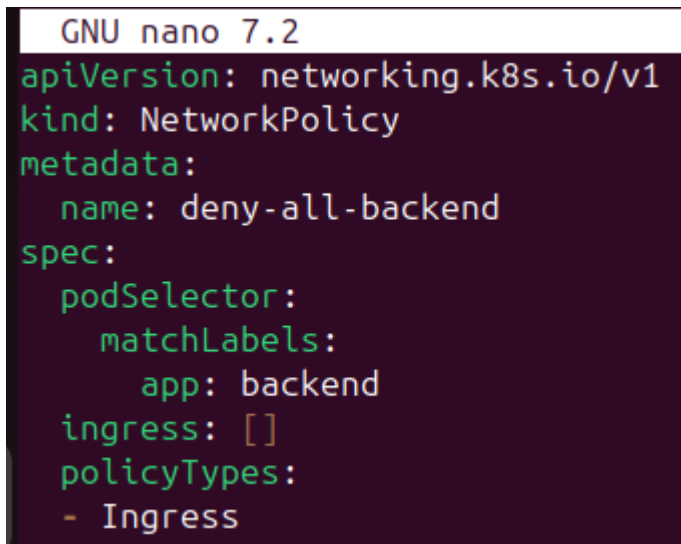
Now we block all incoming traffic to backend.

Create a file `deny-all.yaml`

```
nano deny-all.yaml
```

Paste:

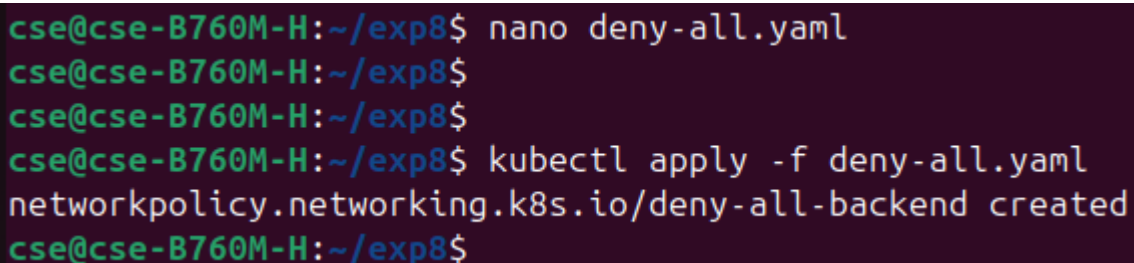
```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress: []
  policyTypes:
  - Ingress
```



```
GNU nano 7.2
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress: []
  policyTypes:
  - Ingress
```

Apply policy

```
kubectl apply -f deny-all.yaml
```



```
cse@cse-B760M-H:~/exp8$ nano deny-all.yaml
cse@cse-B760M-H:~/exp8$
cse@cse-B760M-H:~/exp8$
cse@cse-B760M-H:~/exp8$ kubectl apply -f deny-all.yaml
networkpolicy.networking.k8s.io/deny-all-backend created
cse@cse-B760M-H:~/exp8$
```

```
kubectl get networkpolicy
```

OR

```
kubectl get netpol
```

```
cse@cse-B760M-H:~/exp8$ kubectl get networkpolicy
NAME                POD-SELECTOR  AGE
deny-all-backend   app=backend   5m21s
cse@cse-B760M-H:~/exp8$
```

Step 12: Test Again (After Restriction)

Inside frontend pod:

```
curl <backend-pod-ip>
```

Now it should fail / timeout

This proves:

Traffic is blocked by NetworkPolicy

```
cse@cse-B760M-H:~/exp8$ kubectl exec -it frontend -- /bin/bash
root@frontend:/# curl 10.244.120.68
```

Step 13: Allow Traffic from Frontend Only

Now allow only frontend → backend communication

Create allow policy

```
nano allow-frontend.yaml
```

Paste:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
  policyTypes:
    - Ingress
```

```
GNU nano 7.2
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      app: backend
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: frontend
  policyTypes:
    - Ingress
```

Apply policy

```
kubectl apply -f allow-frontend.yaml
```

```
kubectl get netpol
```

```
cse@cse-B760M-H:~/exp8$ kubectl get netpol
NAME                POD-SELECTOR    AGE
allow-frontend      app=backend     10s
deny-all-backend   app=backend     9m59s
cse@cse-B760M-H:~/exp8$
```

```
kubectl get pods -o wide
```

```
cse@cse-B760M-H:~/exp8$ kubectl get pods -o wide
NAME          READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
backend       1/1    Running   0          11m   10.244.120.68 minikube   <none>           <none>
frontend      1/1    Running   0          12m   10.244.120.67 minikube   <none>           <none>
cse@cse-B760M-H:~/exp8$
```

Access frontend pod

```
kubectl exec -it frontend -- /bin/bash
```

Step 14: Test Again

Inside frontend pod:

```
curl <backend-pod-ip>
```

Now it should work again

```

root@frontend:/# curl 10.244.120.68
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
root@frontend:/#

```

Step 15: Verify Policies

```
kubectl get networkpolicy
```

```

cse@cse-B760M-H:~/exp8$ kubectl get netpol
NAME                                POD-SELECTOR  AGE
allow-frontend                      app=backend   4m12s
deny-all-backend                   app=backend   14m
cse@cse-B760M-H:~/exp8$

```

Step 16: Clean Up

```

kubectl delete pod frontend backend
kubectl delete -f deny-all.yaml
kubectl delete -f allow-frontend.yaml

```

```
cse@cse-B760M-H:~/exp8$ kubectl delete pod frontend backend
pod "frontend" deleted from default namespace
pod "backend" deleted from default namespace
cse@cse-B760M-H:~/exp8$ kubectl delete -f deny-all.yaml
networkpolicy.networking.k8s.io "deny-all-backend" deleted from default namespace
cse@cse-B760M-H:~/exp8$ kubectl delete -f allow-frontend.yaml
networkpolicy.networking.k8s.io "allow-frontend" deleted from default namespace
cse@cse-B760M-H:~/exp8$
```

kubectl get pods

kubectl get netpol

```
cse@cse-B760M-H:~/exp8$ kubectl get pods
No resources found in default namespace.
cse@cse-B760M-H:~/exp8$ kubectl get netpol
No resources found in default namespace.
cse@cse-B760M-H:~/exp8$
```